Les variables

Démonstration 01 du module 4

L'objectif de cette démonstration est de découvrir en quoi, comme tout langage-objet, PowerShell utilise les variables qui permettent de stocker des données dans la mémoire vive de l'ordinateur puis de les réutiliser dans d'autre commande.

Déroulement

- La console PowerShell dispose de variables réservées comme nous avons pu le voir avec `$PSItem`. Nous allons étudier les façons de travailler avec les variables.
- La commande `Get-Variable` permet de connaître toutes les variables disponibles actuellement dans la console PowerShell.

```
PS C:\> Get-Variable |fw -Column 5

$                       ?                       ^                       args                    ConfirmPreference
ConsoleFileName         DebugPreference         Error                   ErrorActionPrefe...     ErrorView
ExecutionContext        false                   FormatEnumeration...    HOME                    Host
InformationPrefer...    input                   MaximumAliasCount       MaximumDriveCount       MaximumErrorCount
MaximumFunctionCount    MaximumHistoryCount     MaximumVariableCount    MyInvocation            NestedPromptLevel
null                    OutputEncoding          PID                     PROFILE                 ProgressPreference
PSBoundParameters       PSCommandPath           PSCulture               PSDefaultParamet...     PSEdition
PSEmailServer           PSHOME                  PSScriptRoot            PSSessionApplica...     PSSessionConfigu...
PSSessionOption         PSUICulture             PSVersionTable          PWD                     ShellId
StackTrace              StorageHistoryCharts    true                    VerbosePreference       WarningPreference
WhatIfPreference
```

- Pour déclarer une nouvelle variable, on utilise le caractère `$` suivi du symbole = puis de la valeur à attribuer à la variable.

```
PS C:\> $a = 1
```

Attention au type de variable ! Le fait d'attribuer la valeur 1 type la variable en `INT`.

Pour obtenir le type de variable, on utilise la méthode `getType()`

```
PS C:\> $a.GetType().fullname
System.Int32
```

Le paramètre `FullName` permet d'afficher le type complet de la variable.

Lorsque l'on souhaite déclarer une variable de type `String` qui va contenir une chaîne de caractères, il faut encadrer les caractères avec des doubles quotes.

```
PS C:\> $b= "1"
PS C:\> $b.GetType().FullName
System.String
```

La variable `$b` est de type `SystemString` car la valeur `1` est encadrée avec des doubles quotes.

© ENI – Tous droits réservés

Les variables vont contenir également des objets, qui par la suite vont être réutilisés dans d'autres traitements.

Pour contenir des objets dans une variable, on déclare la variable puis on assigne un résultat de commande à cette variable.

```
PS C:\> $u=Get-ADUser -Filter *
PS C:\> $u.GetType().fullname
System.Object[]
```

On obtient une variable de type `System.Object` qui contient le résultat de la commande `Get-Aduser`.

Lorsque la variable est déclarée, elle est utilisable dans la console.

Pour visualiser son contenu, il suffit de saisir son nom précédé du `$`.

```
PS C:\> $u


DistinguishedName : CN=Administrator,CN=Users,DC=ad01,DC=lcl
Enabled           : True
GivenName         :
Name              : Administrator
ObjectClass       : user
ObjectGUID        : ecf41ce8-afa0-447f-81f5-ae41c508e63c
SamAccountName    : Administrator
SID               : S-1-5-21-1966697812-3989974639-3621391846-500
Surname           :
UserPrincipalName :
```

La variable peut également servir de conteneur à une saisie dans la console. Elle va être combinée à la commande `Read-Host`

```
PS C:\> $choix = Read-Host "Veuillez saisir votre choix"
Veuillez saisir votre choix: choix1
PS C:\> $choix
choix1
```

La variable peut être utilisable avec la commande `Write-Host` pour afficher le contenu dans une chaîne de caractères avec des outils de mise en forme.

```
PS C:\> Write-Host "Voici le choix de l'utilisateur : $choix"  -ForegroundColor green
Voici le choix de l'utilisateur : choix1
```

La valeur de la variable `$choix` est affichée dans la chaîne de caractères.

Les variables de type `System.Object` peuvent être utilisées pour afficher une ou plusieurs propriétés d'un ou des objets qu'elles contiennent.

On utilise le . à la suite de la variable pour indiquer le nom de la propriété à afficher.

© ENI – Tous droits réservés

```
PS C:\> $s= Get-Service
PS C:\> write-host "Voici les nom des services $s.servicename "
Voici les nom des services AarSvc_128d85 AJRouter ALG AppIDSvc
AppXSvc AssignedAccessManagerSvc AudioEndpointBuilder Audiosrv
```

Il est possible de sélectionner un seul objet dans la variable en lui indiquant une valeur d'index.

```
PS C:\> $s[4].name
Appinfo
PS C:\> $s[4]

Status   Name               DisplayName
------   ----               -----------
Running  Appinfo            Application Information
```

On sélectionne un seul objet avec les [] puis on sélectionne une seule propriété avec le .

Avec les variables de type Objet on peut également se servir des méthodes contenues dans le type d'objet.

```
PS C:\> $d=(get-date).ToString("yyyyMMdd")
PS C:\> $d
20200924
PS C:\> $d.GetType()

IsPublic IsSerial Name                                     BaseType
-------- -------- ----                                     --------
True     True     String                                   System.Object


PS C:\> $d.GetType().fullname
System.String
PS C:\> write-host "nous somme le : $d"
nous somme le : 20200924
```

La variable `$d` utilise la méthode de conversion `ToString` pour convertir une valeur de type `DateTime` renvoyée par la commande `Get-Date`.

En fin de conversion, on obtient une valeur de type string utilisable dans une commande `Write-Host`.

© ENI – Tous droits réservés